ENGINEERING TECHNICAL BRIEF

AI Entity

Applied AI / React / Express / MongoDB / Groq / ONNX embeddings

APPLICATION OVERVIEW

An applied-AI prototype that combines conversation management, document ingestion, semantic knowledge retrieval and configurable context. React handles the interface; Express provides REST and server-sent-event endpoints; MongoDB stores conversations, memories and embedded document chunks.

APPLICATION RESPONSIBILITIES

React / Vite

Chat UI, saved conversations, context controls, memory editor, knowledge-document screens and formatted responses.

Express / Node.js

Conversation, memory and knowledge endpoints, upload processing and Groq-backed response streaming.

MongoDB / Models

Mongoose schemas for conversations, individual memories, and source documents with chunk vectors.

DOCUMENT RETRIEVAL PIPELINE

Ingest and parse

TXT, Markdown, text-based PDF and DOCX are parsed into text. Upload limit: 10 MB.

Chunk and embed

Text is split into 1,000-character chunks with 150-character overlap; normalized ONNX vectors are stored in MongoDB.

Rank and supply context

The query is embedded locally, cosine similarities are calculated over stored chunks, and the top five matches are supplied to the prompt.

MODEL INFERENCE

POST /api/ai/chat uses the Groq SDK with model openai/gpt-oss-20b, temperature 0.7, and stream:true. Express writes SSE data events and a final [DONE] marker. The browser consumes the full streamed response before revealing it with a timed typing effect and saving the conversation.

REQUEST CONTEXT

System messages may include personality instructions, enabled memory, and relevant retrieved passages. Optional conversation history is limited by a configurable recent-message count. The current user prompt is appended after these context messages.

ENGINEERING TECHNICAL BRIEF

Engineering implementation

Source-backed behavior, technical tradeoffs and project boundaries.

DESIGN DECISIONS

Small-scale vector search

The local all-MiniLM-L6-v2 ONNX model generates embeddings using mean pooling and normalization. Vectors are persisted as arrays inside MongoDB documents. Search scans stored chunks and scores them in Node.js; there is no database vector index.

Conversation persistence

Conversations store messages and configurable personality, memory, context and model-settings fields. Frontend settings can be restored or changed, and completed replies are saved through the conversation API.

SECURITY AND PRODUCT BOUNDARIES

Authentication not implemented

Conversation, memory and knowledge CRUD endpoints do not enforce user identity or tenant separation. A public multi-user deployment needs server-side authentication, authorization and access isolation.

Model selection mismatch

The frontend exposes several model options, but the backend inference controller selects openai/gpt-oss-20b. Changing the frontend model setting does not currently switch the model used by the API.

Retrieval scaling and citations

Semantic search scores in application memory and returns document, chunk and score fields. A production retrieval service would need indexing, access-scoped search and stronger source attribution controls.

Verification status

The inspected repository contains application code and dependencies. No committed automated tests, CI workflow or reproducible end-to-end benchmark was found. Live Groq and database flows were not executed during this review.

SOURCE REFERENCE

backend/src/controllers/aiController.js
backend/src/controllers/knowledgeController.js
backend/src/services/embeddingService.js
backend/src/services/knowledgeSearchService.js
backend/src/models/Conversation.js
frontend/src/App.jsx / frontend/src/api/ai.js